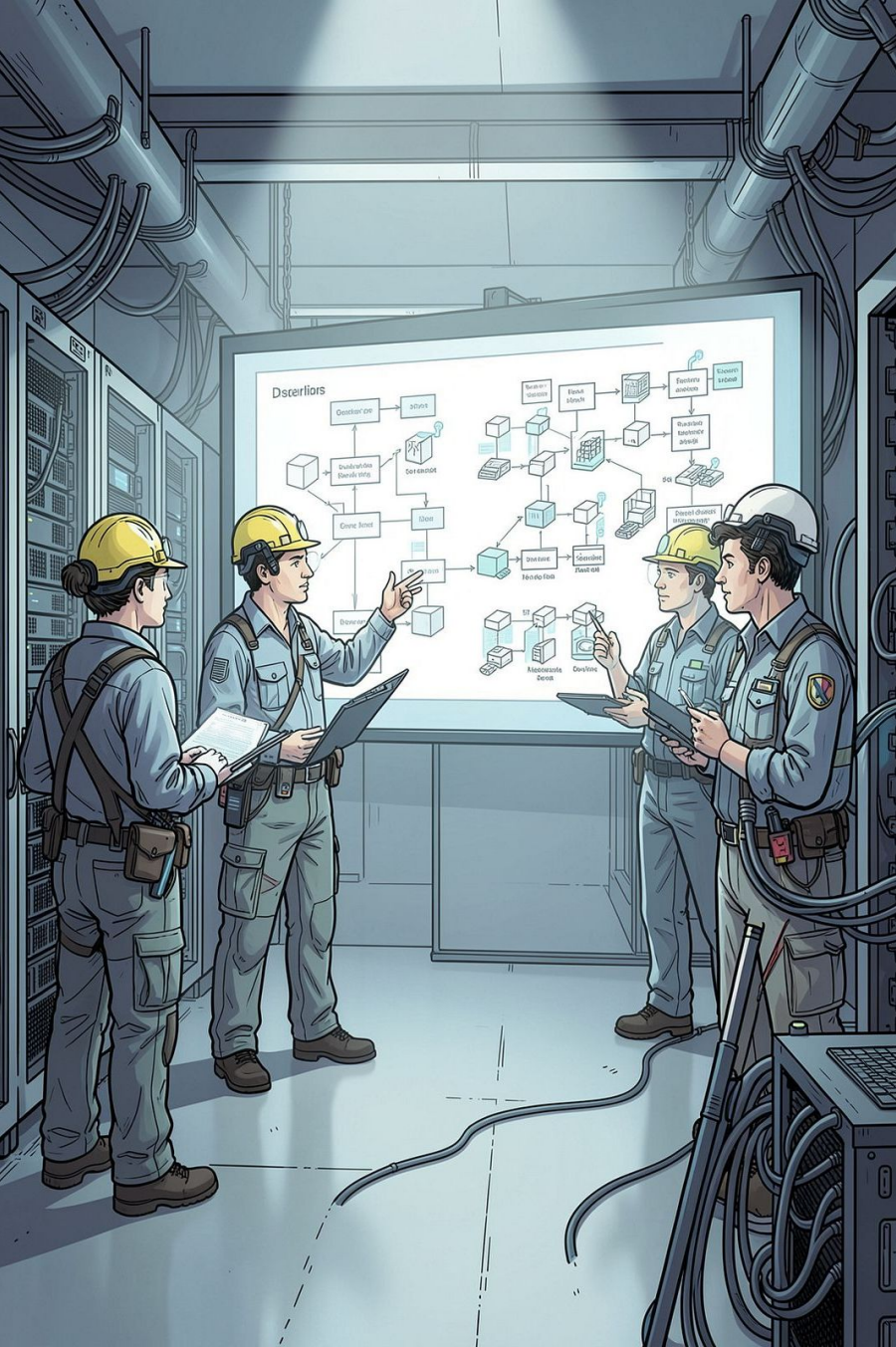




Lumen Academy — Database Schema & Architecture

Assessment engine and LMS built for NEET, JEE Main, and JEE Advanced. PostgreSQL 16 on Supabase, Prisma v6.14.0. 36 Prisma models across 11 domain modules. Security-first design: server-controlled clocks, IDOR-proof authorization, and zero answer leakage.



Core Architectural Principles — 4 Non-Negotiable Schema Laws

Zero-Leak Answer Keys

Fields like `isCorrect` and `numericalAnswer` live only on server-side snapshots and are never selected or returned before attempt submission.

Server-Controlled Clocks

`startedAt`, `deadlineAt`, and `sectional lockedAt` are authoritative on server time – client clocks are untrusted.

IDOR-Proof Authorization

Every student row ties to `userId` or to a parent entity owned by that `userId` to prevent access-by-id guessing.

Immutable Result Freeze

`AttemptQuestion` snapshots question stem, options, marking rules, and answer keys at submit time – edits never touch historical scores.

11 Domain Modules — 36 Prisma Models

Concise module list and representative Prisma models for each domain. This structure guides ownership, RBAC boundaries, and rollup responsibilities for analytics and grading.



01 — Identity & Security

User, StudentProfile,
UserPreference,
AuditLog



02 — Syllabus Taxonomy

Subject, Unit, Topic



03 — Question Bank

Question, Option –
supports 5 interaction
formats



Identity & Profile Architecture

Primary user model maps to Supabase auth; optional fields for email/phone and role-based defaults. StudentProfile feeds the AIR rank predictor via categorical inputs.

AIR Rank Inputs

`StudentProfile.category` (GENERAL, EWS, OBC_NCL, SC, ST) and `StudentProfile.state` – used to calculate home-state quotas and cutoffs.

Audit & Compliance

`AuditLog` stores hashed IP and privileged-action records for forensic review and compliance.

Question Bank — 5 Interaction Formats & Review Lifecycle



SINGLE_CORRECT

Standard 4-option MCQs used for NEET/JEE Main.



MULTIPLE_CORRECT

Multiple answers with partial marking contracts for advanced exams.



NUMERICAL

Integer/decimal inputs graded with `numericalTolerance` metadata.



MATCHING

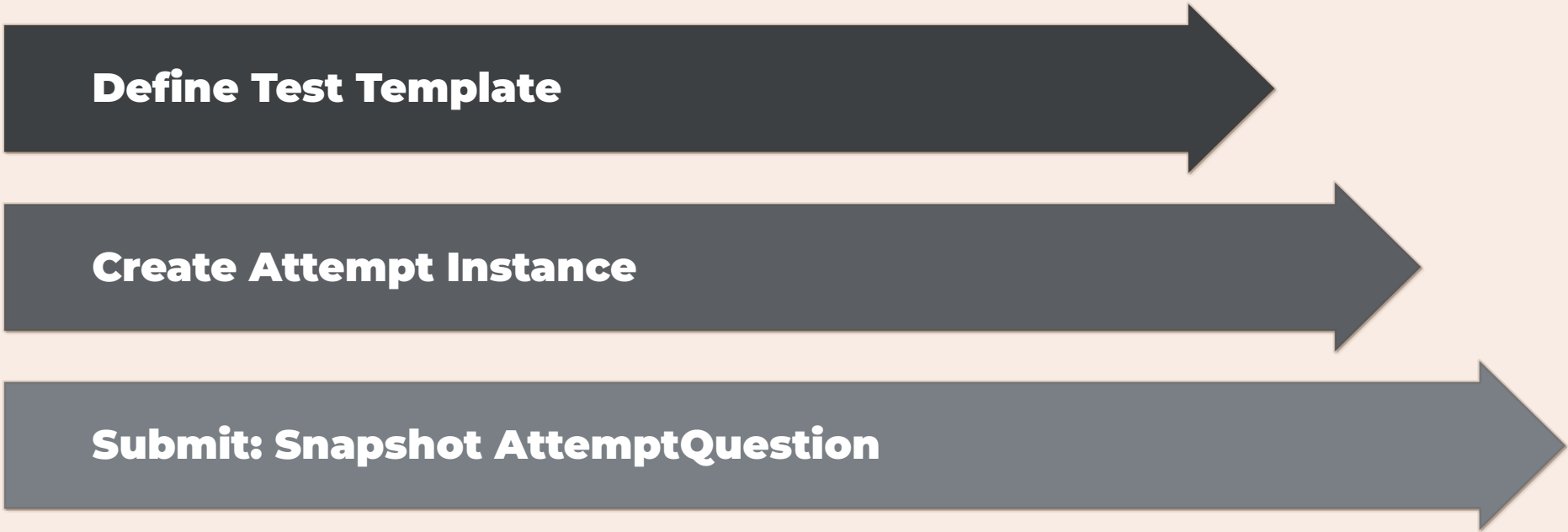
Left/right matrix mappings with validation rules and UI hints.



ASSERTION_REASON

Paired statements with combined correctness logic and partial scoring.

Editorial pipeline: DRAFT → PENDING_REVIEW → PUBLISHED → RETIRED with human critic paths and versioning.

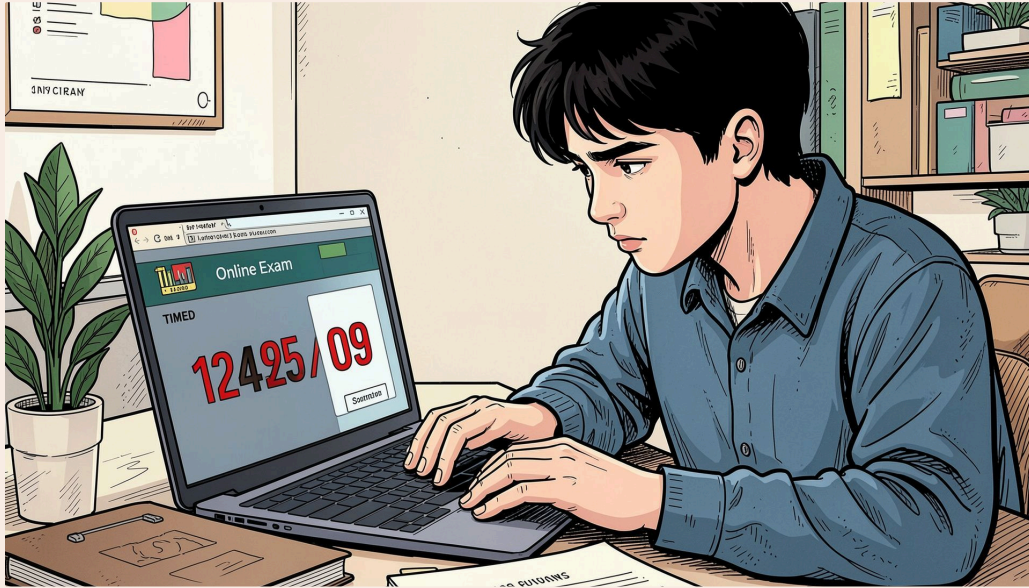


Define Test Template

Create Attempt Instance

Submit: Snapshot AttemptQuestion

Separation of template and instance preserves templates for reuse while attempts hold authoritative, immutable snapshots used for grading and audit.



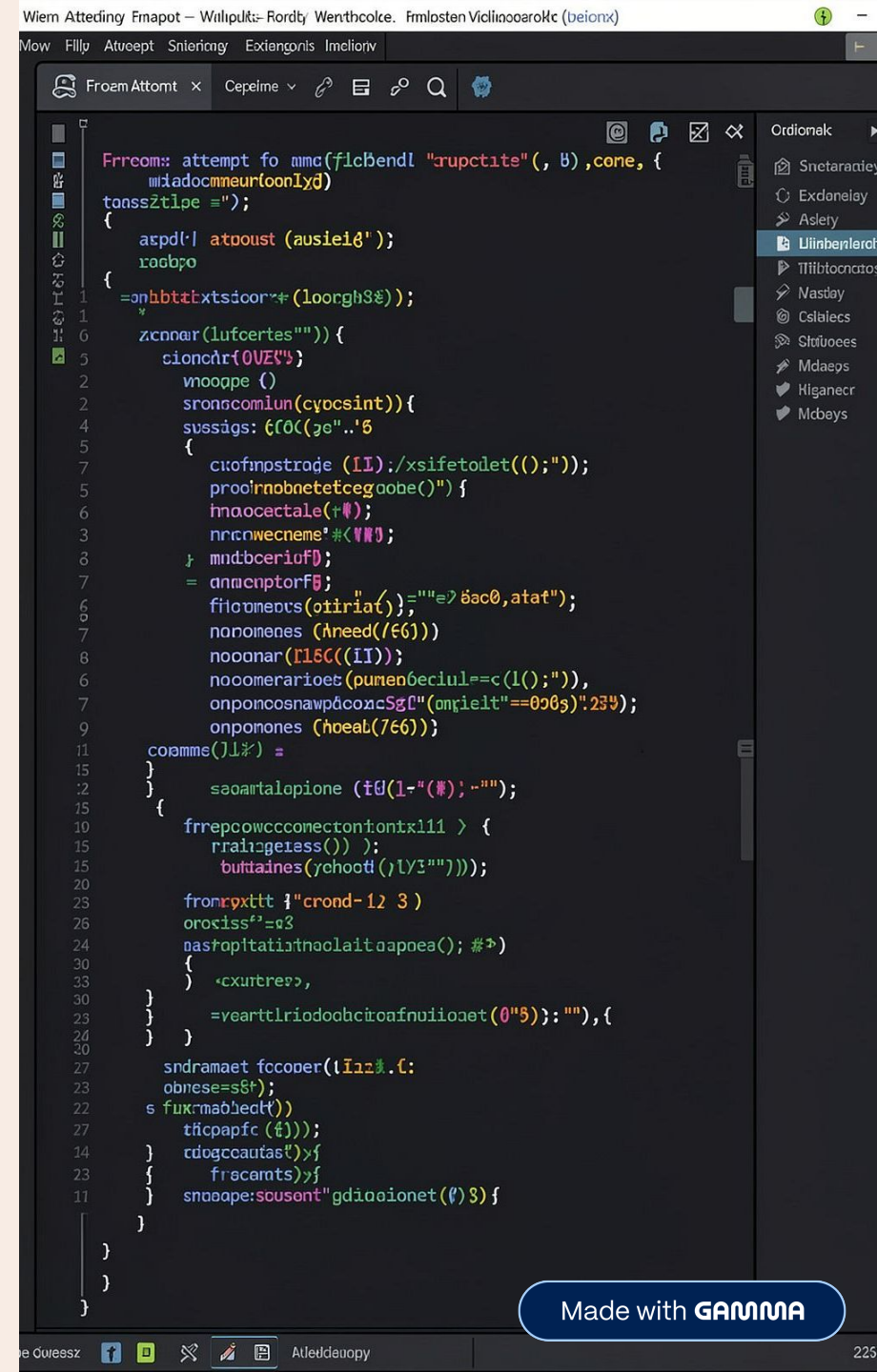
Attempt Engine — Sectional Timing & Autosave

- **Sectional Clock Enforcement:** Independent `AttemptSection.lockedAt` values let sections lock separately (required for JEE sectional rules).
- **Autosave Resilience:** `AttemptQuestion` uses upsert semantics keyed by `(attemptId, questionId)` to support intermittent connectivity.
- **Scoring:** Server computes marks at submission time using snapshotted rules; no client-side scoring trusted.

Law 4 — Immutable Snapshotting (Example)

```
{
  "attemptId": "att-9901",
  "questionId": "q-4412",
  "contentSnapshot": { "stemEn": "What is unit vector magnitude?" },
  "correctOptionIdsSnapshot": ["opt-a-uuid"],
  "marksSnapshot": 4,
  "negativeMarksSnapshot": 1,
  "marksAwarded": 4,
  "isCorrect": true
}
```

📄 Business value: Corrections to live Question records never retroactively change historical student scores or transcripts.





LMS, Gamification & Productivity Tools

Integrated content and retention features designed to increase daily active use and learning efficacy.



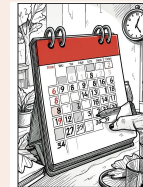
Lesson Video

Timestamps, bookmarks, and watch-percentage tracking (`LessonVideo`, `VideoTimestamp`, `LessonProgress`).



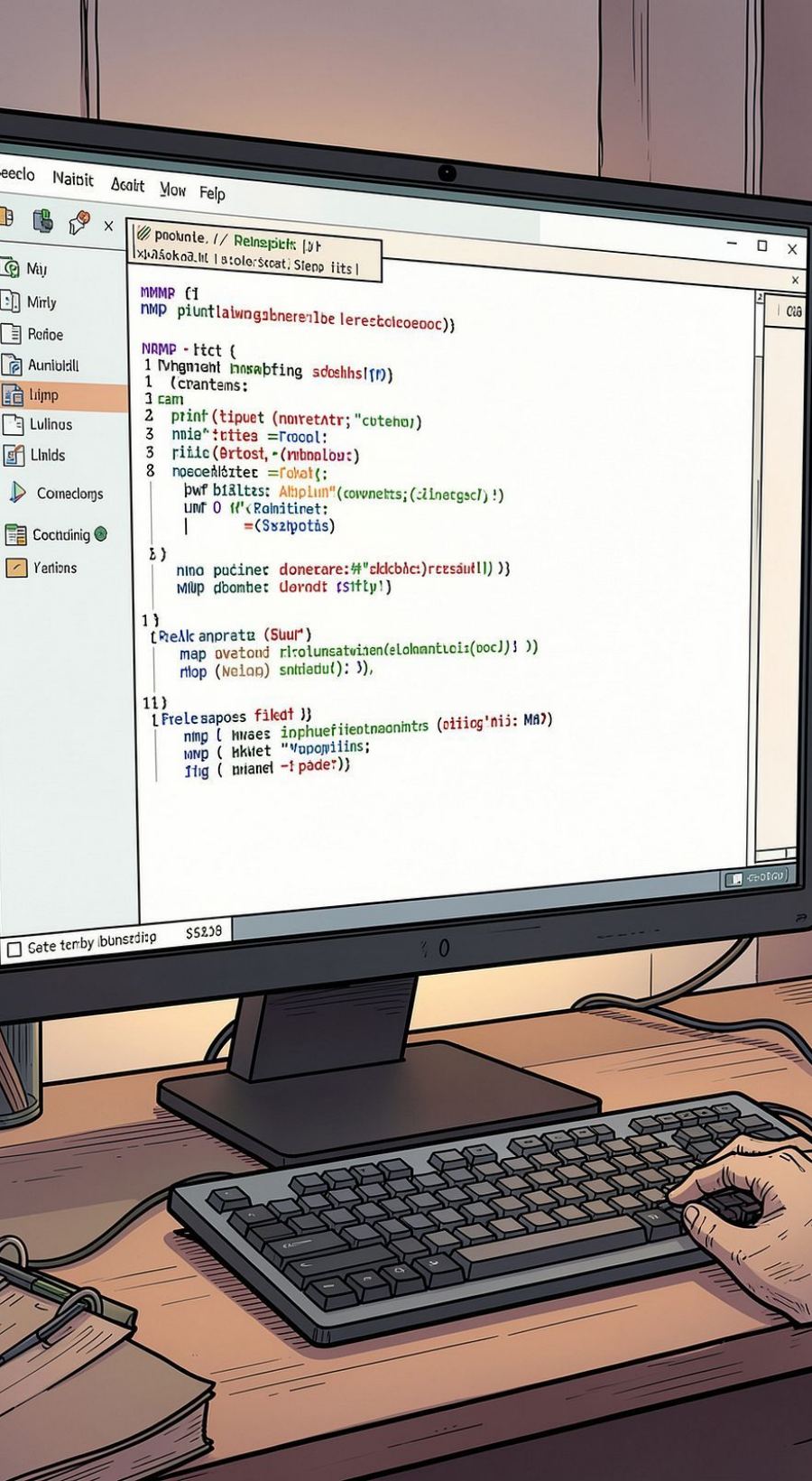
Gamification

Streak, Badge, DailyGoal, push Notification queue.



Productivity Suite

PomodoroSession, StudyPlan (AI-generated timetables), Flashcard, RevisionNote.



Developer Operations & Production Status

CLI and CI workflows ensure schema validity, type safety, and deployable artifacts.

DB Validation npm run db:validate – schema syntax and relation checks.	Client Generation npm run db:generate – generate type-safe @prisma/client.	Model Verification npm run db:verify- all – populate and sanity-check all 36 models.
Studio & Typecheck npm run db:studio; npm run typecheck passed clean on PostgreSQL 16 (Supabase).		

Production note: All models validated and deployed with zero TypeScript errors. Follow-up items: formalize backup/restore SLAs, run periodic snapshot audits, and codify emergency rollback procedures.